COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - DSA	**SECTION:**	CPE21S1
DATE PERFORMED:	09/15/2026	**DATE SUBMITTED:**	09/15/2026
NAME:	Aguila, Daniel C.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. HOA 8.1 Sorting Algorithms Pt2

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create C++ code for implementation of shell, merge and quick sort

Code + Console Screenshot

main.cpp > ...

```cpp
//Aguila, Daniel C. CPE21S1
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sortingp.h"

const int maxSize = 100;

void displayAll(int arr[]);
void copyArr(int source[], int destination[]);

int main() {
    int unsortedArr[maxSize];
    int copiedArr[maxSize];
    srand(time(0));
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand() % 100;
    }

    std::cout << "Unsorted Array: ";
    displayAll(unsortedArr);

    std::cout << "\nShell Sort: ";
    copyArr(unsortedArr, copiedArr);
    shellSort(copiedArr, maxSize);
    displayAll(copiedArr);

    std::cout << "\nMerge Sort: ";
    copyArr(unsortedArr, copiedArr);
    mergeSort(copiedArr,0 , maxSize - 1);
    displayAll(copiedArr);

```

Observations

The program that was written is one which takes into consideration an array with 100 randomly generated numbers. The original order of these numbers was the order in which they were first stored and shown, i.e., these elements did not start off sorted in the order of increasing magnitude. It was important

	to keep the array unsorted, as it would allow me to test all three sorting algorithms that were considered in the program with the same set of conditions, which was an unsorted array. The sorting algorithms that were tested in this program include **Shell Sort, Merge Sort, and Quick Sort**. By sorting the values in ascending order, I got to know how each of these algorithms work on a given amount and type of data.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	
Observations	Shell Sort has proved successful in sorting the randomly generated 100 numbers in an ascending order. While the adjacent elements are not compared, Shell sort will at first compare the elements that have a specific gap. The gap helps in ensuring that the elements that are very far from their actual position get shifted to near their position. With the process of sorting taking place, the gap slowly gets reduced hence the algorithm can now conduct further comparisons on the elements. After the gap reduces significantly, the elements are sorted using the comparisons of closer elements. The elements are slowly sorted until they end up in the right places hence forming a series from the smallest number to the largest number.

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```cpp
//merge sort helper
template <typename T>
void merge(T arr[], int left, int mid, int right){
    int n1 = mid - left + 1;
    int n2 = right - mid;

    T* leftArr = new T[n1];
    T* rightArr = new T[n2];

    for(int i = 0; i < n1; i++){
        leftArr[i] = arr[left + i];
    }
    for(int j = 0; j < n2; j++){
        rightArr[j] = arr[mid + 1 + j];
    }

    int i = 0 , j = 0, k = left;
    while(i < n1 && j < n2){
        if(leftArr[i] <= rightArr[j]){
            arr[k] = leftArr[i];
            i++;
        }
        else{
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }
```

```cpp
    while(i < n1){
        arr[k] = leftArr[i];
        i++;
        k++;
    }
    while(j < n2){
        arr[k] = rightArr[j];
        j++;
        k++;
    }
}

//merge sort
template <typename T>
void mergeSort(T arr[], int left, int right){
    if(left < right){
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
```

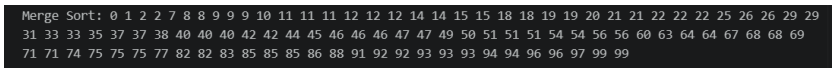
	
Observations	The Merge Sort algorithm was successful in sorting the 100 randomly generated numbers in ascending order from the smallest number to the largest. The sorting algorithm uses the divide and conquer approach to sort the initial array by first splitting it into smaller parts until each part has only one element in it. Since one element is automatically considered sorted, the next step of the algorithm is to start merging the small arrays while sorting the elements inside. The merging procedure makes sure that smaller elements are merged first in order for each small array to be sorted. These steps of merging and dividing are performed until all the small arrays merge into one large sorted array.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	

```
73  //quicksort
74  template <typename T>
75  int partition(T arr[], int low, int high){
76      T pivot = arr[high];
77      int i = low - 1;
78
79      for(int j = low; j < high; j++){
80          if(arr[j] <= pivot){
81              i++;
82              std::swap(arr[i], arr[j]);
83          }
84      }
85      std::swap(arr[i+1], arr[high]);
86      return i + 1;
87  }
88
89  template <typename T>
90  void quickSort(T arr[], int low, int high){
91      if(low < high){
92          int pivotIndex = partition(arr, low, high);
93          quickSort(arr, low, pivotIndex -1);
94          quickSort(arr, pivotIndex+1, high);
95      }
96  }
```

```
    std::cout << "\nQuick Sort: ";
    copyArr(unsortedArr, copiedArr);
    quickSort(copiedArr,0 , maxSize - 1);
    displayAll(copiedArr);


    return 0;
}


void displayAll(int arr[]){
    for(int i = 0; i < maxSize; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

void copyArr(int source[], int destination[]){
    for(int i = 0; i < maxSize; i++){
        destination[i] = source[i];
    }
}
```

```
Quick Sort: 0 1 2 2 7 8 8 9 9 9 10 11 11 11 12 12 12 14 14 15 15 18 18 19 19 20 21 21 22 22 22 25 26 26 29 29
31 33 33 35 37 37 38 40 40 40 42 42 44 45 46 46 46 47 47 49 50 51 51 51 54 54 56 56 60 63 64 64 67 68 68 69
71 71 74 75 75 75 77 82 82 83 85 85 85 86 88 91 92 92 93 93 93 94 94 96 96 97 99 99
```

Observations	Quick Sort was able to sort the 100 random numbers in an ascending order, from the lowest number to the highest. In the sorting process, Quick Sort first chose a certain number from the array as the **pivot**. The other numbers would then be compared to the pivot and separated according to whether they were smaller or bigger than the pivot number. In doing so, there would be the formation of smaller arrays from the main array which could be sorted separately. Quick Sort would then choose the pivots of these smaller arrays and compare and separate them again using the same method. The process would be repeated until the arrays were small enough and the numbers sorted accordingly. After the process of comparing and sorting had been done, the small arrays would be brought together to form one sorted array comprising of all the 100 numbers.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO B: Solve given data sorting problems using appropriate basic sorting algorithms

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

MAIN.CPP (PROBLEM 1)

main.cpp > ...

```cpp
//Aguila, Daniel C. CPE21S1
#include <iostream>
#include "SortingAlgorithms.h"

int main()
{
    int arr[] = {8, 3, 7, 4, 9, 2, 6};
    int size = 7;

    std::cout << "Original Array: ";
    displayArray(arr, size);

    int pivot = 7;

    int leftSublist[7];
    int rightSublist[7];

    int leftSize = 0;
    int rightSize = 0;

    for (int i = 0; i < size; i++)
    {
        if (arr[i] < pivot)
        {
            leftSublist[leftSize] = arr[i];
            leftSize++;
        }
        else if (arr[i] > pivot)
        {
            rightSublist[rightSize] = arr[i];
            rightSize++;
        }
    }
```

```
            rightSublist[rightSize] = arr[i];
            rightSize++;
        }
    }

    std::cout << "\nPivot: " << pivot << std::endl;

    std::cout << "Left Sublist before sorting: ";
    displayArray(leftSublist, leftSize);

    std::cout << "Right Sublist before sorting: ";
    displayArray(rightSublist, rightSize);

    insertionSort(leftSublist, leftSize);
    insertionSort(rightSublist, rightSize);

    std::cout << "\nLeft Sublist after Insertion Sort: ";
    displayArray(leftSublist, leftSize);

    std::cout << "Right Sublist after Insertion Sort: ";
    displayArray(rightSublist, rightSize);

    std::cout << "\nFinal Sorted Array: ";

    for (int i = 0; i < leftSize; i++)
    {
        std::cout << leftSublist[i] << " ";
    }

    std::cout << pivot << " ";

    for (int i = 0; i < rightSize; i++)
    {
        std::cout << rightSublist[i] << " ";
    }
    {
        std::cout << rightSublist[i] << " ";
    }

    std::cout << std::endl;

    return 0;
}
```

SORTINGALGORITHMS.H (PROBLEM 1)

```
#ifndef SORTINGALGORITHMS_H
#define SORTINGALGORITHMS_H

#include <iostream>

void insertionSort(int arr[], int size)
{
    for (int i = 1; i < size; i++)
    {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}

void displayArray(int arr[], int size)
{
    for (int i = 0; i < size; i++)
    {
        std::cout << arr[i] << " ";
    }

    std::cout << std::endl;
}

#endif
```

OUTPUT (PROBLEM 1)

```
Original Array: 8 3 7 4 9 2 6

Pivot: 7
Left Sublist before sorting: 3 4 2 6
Right Sublist before sorting: 8 9

Left Sublist after Insertion Sort: 2 3 4 6
Right Sublist after Insertion Sort: 8 9

Final Sorted Array: 2 3 4 6 7 8 9
```

In this program, I utilized different functions so that the sorting process would be well-organized. The functions insertionSort() was meant to arrange the left and right sublists in ascending order, whereas the displayArray() function was utilized to display the values before and after sorting. The original array was dealt with in the main() function, where the pivot value was chosen as 7 and values were divided into left and right sublists depending on whether they are smaller or larger than the pivot. The use of functions helped organize the program because there was a particular task for each function and not everything was placed in the main() function.

MAIN.CPP (PROBLEM 2)

```cpp
//Aguila, Daniel C.
#include <iostream>
#include "SortingAlgorithms.h"

int main()
{
    int original[] = {
        4, 34, 29, 48, 53,
        87, 12, 30, 44, 25,
        93, 67, 43, 19, 74
    };

    int size = 15;

    // Create separate arrays for each sorting algorithm
    int mergeArray[15];
    int quickArray[15];
```

```
    int quickArray[15];

    for (int i = 0; i < size; i++)
    {
        mergeArray[i] = original[i];
        quickArray[i] = original[i];
    }

    std::cout << "Original Array: ";
    displayArray(original, size);

    // Apply Merge Sort
    mergeSort(mergeArray, 0, size - 1);

    std::cout << "\nAfter Merge Sort: ";
    displayArray(mergeArray, size);

    // Apply Quick Sort
    quickSort(quickArray, 0, size - 1);

    std::cout << "After Quick Sort: ";
    displayArray(quickArray, size);
```

```
    std::cout << "After Quick Sort: ";
    displayArray(quickArray, size);

    std::cout << "\nTime Complexity:" << std::endl;
    std::cout << "Merge Sort: O(n log n)" << std::endl;
    std::cout << "Quick Sort: O(n log n) average case" << std::endl;
    std::cout << "Quick Sort: O(n^2) worst case" << std::endl;

    return 0;
}
```

SORTINGALGORITHMS.H (PROBLEM 2)

```
#ifndef SORTINGALGORITHMS_H
#define SORTINGALGORITHMS_H

#include <iostream>

// Merge two sorted portions of the array
void merge(int arr[], int left, int middle, int right)
{
```

```
7   void merge(int arr[], int left, int middle, int right)
8   {
9       int n1 = middle - left + 1;
10      int n2 = right - middle;
11
12      int* L = new int[n1];
13      int* R = new int[n2];
14
15      for (int i = 0; i < n1; i++)
16      {
17          L[i] = arr[left + i];
18      }
19
20      for (int j = 0; j < n2; j++)
21      {
22          R[j] = arr[middle + 1 + j];
23      }
24
25      int i = 0;
26      int j = 0;
27      int k = left;
28
29      while (i < n1 && j < n2)
30      {
```

```
29      while (i < n1 && j < n2)
30      {
31          if (L[i] <= R[j])
32          {
33              arr[k] = L[i];
34              i++;
35          }
36          else
37          {
38              arr[k] = R[j];
39              j++;
40          }
41
42          k++;
43      }
44
45      while (i < n1)
46      {
47          arr[k] = L[i];
48          i++;
49          k++;
50      }
51
52      while (j < n2)
53      {
```

```
53      {
54          arr[k] = R[j];
55          j++;
56          k++;
57      }
58
59      delete[] L;
60      delete[] R;
61  }
62
63  // Merge Sort
64  void mergeSort(int arr[], int left, int right)
65  {
66      if (left < right)
67      {
68          int middle = left + (right - left) / 2;
69
70          mergeSort(arr, left, middle);
71          mergeSort(arr, middle + 1, right);
72
73          merge(arr, left, middle, right);
74      }
75  }
76
77  // Partition for Quick Sort
78  int partitionArray(int arr[], int low, int high)
79  {
80      int pivot = arr[high];
```

```
80      int pivot = arr[high];
81      int i = low - 1;
82
83      for (int j = low; j < high; j++)
84      {
85          if (arr[j] < pivot)
86          {
87              i++;
88
89              int temp = arr[i];
90              arr[i] = arr[j];
91              arr[j] = temp;
92          }
93      }
94
95      int temp = arr[i + 1];
96      arr[i + 1] = arr[high];
97      arr[high] = temp;
```

```
96      arr[i + 1] = arr[high];
97      arr[high] = temp;
98
99      return i + 1;
100 }
101
102 // Quick Sort
103 void quickSort(int arr[], int low, int high)
104 {
105     if (low < high)
106     {
107         int pivotIndex = partitionArray(arr, low, high);
108
109         quickSort(arr, low, pivotIndex - 1);
110         quickSort(arr, pivotIndex + 1, high);
111     }
112 }
113
114 // Display Array
115 void displayArray(int arr[], int size)
116 {
117     for (int i = 0; i < size; i++)
118     {
119         std::cout << arr[i] << " ";
120     }
121
122     std::cout << std::endl;
123 }
124
125 #endif
```

OUTPUT (PROBLEM 2)

```
[Running] cd "c:\Users\daniel\OneDrive\CPE21S1 Aguila\HOA8.1 Supp\pr
&& "c:\Users\daniel\OneDrive\CPE21S1 Aguila\HOA8.1 Supp\prob2\"prob2
Original Array: 4 34 29 48 53 87 12 30 44 25 93 67 43 19 74

After Merge Sort: 4 12 19 25 29 30 34 43 44 48 53 67 74 87 93
After Quick Sort: 4 12 19 25 29 30 34 43 44 48 53 67 74 87 93

Time Complexity:
Merge Sort: O(n log n)
Quick Sort: O(n log n) average case
Quick Sort: O(n^2) worst case

[Done] exited with code=0 in 0.613 seconds
```

In this code, I defined various functions for different phases of sorting. The mergeSort() function split the array into smaller subarrays, whereas the merge() function joined the subarrays back in ascending order. On the other hand, Quick Sort was done through the partitionArray() and quickSort() functions, where the former selected a pivot value and sorted the elements on its basis while the latter sorted the subarrays thus formed. In addition to the above-mentioned functions, I used the displayArray() function to display both the original and sorted arrays. Through the use of the aforementioned functions, I ensured better organization of the program, which enabled me to carry out better comparisons of the two sorting algorithms on the same list of numbers.

CONCLUSION

Provide the following:

- **Summary of lessons learned**
- **Analysis of the procedure**
- **Analysis of the supplementary activity**
- **Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?**

Through this procedure and supplementary activity, I have learned the manner in which various sorting algorithms arrange data. In addition, through each algorithm, I have understood how the algorithm arranges its values in increasing order. Through the sorting process, I was able to learn how Shell Sort, Merge Sort, Quick Sort, and Insertion Sort were executed through comparison, division, and rearrangement of values. Through the process, I have learned how functions are used in organizing a program as each of these functions performs particular tasks, such as sorting, partitioning, merging or displaying the arrays. The exercises also helped me apply what I had learned through use of Insertion Sort to sort the sublists that result

from Quick Sort and also through comparison of time complexities for Merge Sort and Quick Sort. Overall, I believe that I performed quite well in the activity as I was able to follow the sorting process as well as how the functions worked. However, there is need to further improve my understanding of recursion, partitioning and time complexities so that I am able to write and analyze sorting programs easily without much trial and error.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **I used AI for code assistance, correction of grammar, and more information on variables**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **This AI was used in good purpose. It helped me when I got confused on the codes, it simply tells the process**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.